Le store et son utilisation dans Panier

Le **store** est l'endroit où Redux conserve les données partagées de ton application. Dans notre exemple, il contient la quantité d'articles du panier.

1. À quoi ressemble l'état du store ?

```
{
  panier: {
    quantite: 0
  }
}
```

`panier` regroupe les données du panier. `quantite` contient le nombre d'articles.

2. Comment crée-t-on le store ?

Dans `src/redux-example/store/store.js` :

```
import { configureStore } from "@reduxjs/toolkit";
import panierReducer from "./panierSlice.js";

export const store = configureStore({
  reducer: {
    panier: panierReducer
  }
});
```

`configureStore` crée le store. La clé `panier` définit le nom de cette partie de l'état. Le `panierReducer` définit son état initial et comment les actions le modifient.

Le chemin pour lire le nombre d'articles est donc `state.panier.quantite`.

3. Comment les composants y accèdent-ils ?

Dans `src/redux-example/main.jsx`, on entoure App avec `Provider` :

```
<Provider store={store}>
  <App />
</Provider>
```

Provider rend le store accessible à tous les composants situés en dessous de lui, dont Navigation et Panier. Il évite de transmettre le store manuellement à chaque niveau par des props.

Lire la donnée dans un composant

4. Comment Panier lit-il la quantité ?

Dans `src/redux-example/components/Panier.jsx`, la lecture fonctionne ainsi (version simplifiée de l'affichage) :

```
import { useSelector } from "react-redux";

export default function Panier() {
  const quantite = useSelector(
    state => state.panier.quantite
  );

  return <p>Panier : {quantite} article(s)</p>;
}
```

`useSelector` est un hook fourni par React Redux. Il retrouve le store rendu accessible par Provider et sélectionne la donnée dont le composant a besoin.

- `state` représente tout l'état du store.
- `state.panier` récupère la partie panier.
- `state.panier.quantite` récupère le nombre d'articles.
- Cette valeur est placée dans la variable `quantite` pour l'afficher.

> **À retenir :** Panier n'importe pas directement le store et ne reçoit pas la quantité en prop. Il utilise `useSelector` pour lire le store fourni par `Provider`.

5. Pourquoi l'affichage se met-il à jour ?

`useSelector` abonne aussi Panier aux changements du store. Quand la quantité sélectionnée change, React affiche la nouvelle valeur dans Panier.

```
Clic sur « Ajouter au panier » dans Produit
                  ↓
        dispatch(ajouter())
                  ↓
Le reducer augmente la quantité dans le store
                  ↓
useSelector détecte la nouvelle quantité
                  ↓
      Panier affiche la nouvelle valeur
```

Pour modifier l'état, Produit envoie une action avec `dispatch`. Pour le lire, Panier utilise `useSelector`. Navigation peut toujours contenir Panier, sans transmettre la quantité.

> **Les trois rôles :**
> Store : conserve les données partagées.
> Provider : donne accès au store dans l'arbre React.
> useSelector : lit une donnée et suit ses changements.

6. Quel est le rôle de panierSlice.js ?

Un **slice** regroupe la définition d'une partie de l'état Redux et les règles permettant de la modifier. Ici, cette partie correspond au panier.

Voici le code de `src/redux-example/store/panierSlice.js` :

```
import { createSlice } from "@reduxjs/toolkit";

const panierSlice = createSlice({
  name: "panier",
  initialState: { quantite: 0 },
  reducers: {
    ajouter(state) {
      state.quantite += 1;
    },
  },
});

export const { ajouter } = panierSlice.actions;
export default panierSlice.reducer;
```

- `createSlice` génère le reducer du slice et les fonctions qui créent ses actions.
- `name: "panier"` sert de préfixe aux types des actions générées, par exemple `"panier/ajouter"`.
- `initialState` définit les données de départ : la quantité vaut 0.
- `reducers` contient les règles de modification. Ici, `ajouter` augmente la quantité de 1.

Dans ce reducer, `state` représente uniquement l'état du panier : `{ quantite: 0 }`. Dans `useSelector`, `state` représente l'état complet du store : `{ panier: { quantite: 0 } }`.

> **Pourquoi peut-on écrire state.quantite += 1 ?**
> Redux Toolkit utilise Immer : on travaille sur un brouillon, et Immer produit le nouvel état sans modifier l'ancien. Cette écriture est prévue dans les reducers de createSlice ; on ne modifie pas directement le store depuis un composant.

Relier le slice, le store et les composants

7. Pourquoi exporter ajouter et le reducer ?

Les deux exports ont des rôles différents :

```
export const { ajouter } = panierSlice.actions;
```

`ajouter` est une **fonction qui crée une action**. Appeler `ajouter()` crée un objet dont le type est `"panier/ajouter"`. Cet appel seul ne modifie pas le store : il faut envoyer l'action avec `dispatch`.

```
export default panierSlice.reducer;
```

Cet export fournit le **reducer**, la fonction qui calcule le nouvel état du panier selon l'action reçue. Dans `store.js`, on l'importe sous le nom `panierReducer` :

```
import panierReducer from "./panierSlice.js";

export const store = configureStore({
  reducer: { panier: panierReducer }
});
```

La clé `panier` dans cette configuration définit le chemin `state.panier`. Le `name` du slice définit le préfixe des actions ; ce sont deux rôles distincts, même si nous utilisons le même mot ici.

8. Comment Produit utilise-t-il l'action ?

Version simplifiée de `components/Produit.jsx` :

```
import { useDispatch } from "react-redux";
import { ajouter } from "../store/panierSlice.js";

export default function Produit() {
  const dispatch = useDispatch();

  return (
    <button onClick={() => dispatch(ajouter())}>
      Ajouter au panier
    </button>
  );
}
```

Au clic, `ajouter()` crée l'action, puis `dispatch` l'envoie au store. Le reducer du panier traite cette action et augmente la quantité. Panier, abonné à cette quantité avec `useSelector`, affiche la nouvelle valeur.

> **Les fichiers à relier mentalement :**
> panierSlice.js : définit les données de départ, les actions et les règles de modification.
> store.js : enregistre le reducer sous la clé panier.
> main.jsx : fournit le store via Provider.
> Produit.jsx : envoie l'action ajouter.
> Panier.jsx : lit state.panier.quantite avec useSelector.

Documentation : Guide officiel Redux — Quick Start.